

Fl.Event.Handling

This library provides the core abstractions and implementations for event handling using a functional approach built on top of [LanguageExt](#).

Abstractions

`IEventHandler<TEvent>`

Represents a single handler responsible for reacting to an event of type `TEvent`.

```
public interface IEventHandler<TEvent> where TEvent : class
{
    EitherAsync<Error, Unit> HandleAsync(TEvent evt);
}
```

Implement this interface to define the logic that should execute when a specific event is received. Multiple implementations can be registered for the same `TEvent` and composed through an `IEventProcessor<TEvent>`.

`IEventProcessor<TEvent>`

Represents an orchestrator that coordinates one or more `IEventHandler<TEvent>` instances to process an event.

```
public interface IEventProcessor<TEvent> where TEvent : class
{
    EitherAsync<Error, Unit> ProcessAsync(TEvent evt);
}
```

Both methods return `EitherAsync<Error, Unit>`: an asynchronous computation that yields `Unit` on success or an `Error` on failure, keeping the processing pipeline fully functional and free of thrown exceptions.

Processors

`SequentialEventProcessor<T>`

Dispatches an event to all registered `IEventHandler<T>` instances **one at a time**, in registration order. If any handler returns an `Error`, processing stops immediately and the error is propagated.

```
var processor = new SequentialEventProcessor<MyEvent>(handlers);
EitherAsync<Error, Unit> result = processor.ProcessAsync(myEvent);
```

ParallelEventProcessor<T>

Dispatches an event to all registered `IEventHandler<T>` instances **concurrently**, awaiting all of them before returning. Use this when handlers are independent and latency matters.

```
var processor = new ParallelEventProcessor<MyEvent>(handlers);
EitherAsync<Error, Unit> result = processor.ProcessAsync(myEvent);
```

Decorators

Both decorators implement `IEventProcessor<TEvent>` and wrap an existing processor, following the decorator pattern so they can be freely composed.

ExceptionCatcherEventProcessor<TEvent>

Wraps an inner processor and converts any unhandled exception thrown during processing into an `Error` value, preventing exceptions from escaping the functional pipeline.

```
IEventProcessor<MyEvent> safe = new ExceptionCatcherEventProcessor<MyEvent>
(innerProcessor);
```

LoggingEventProcessor<TEvent>

Wraps an inner processor and logs the outcome of each processing call using a [Serilog](#) `ILogger`.

```
IEventProcessor<MyEvent> logged = new LoggingEventProcessor<MyEvent>
(innerProcessor, logger);
```

Composition example

Decorators can be stacked to combine concerns:

```
IEventProcessor<MyEvent> processor =
    new LoggingEventProcessor<MyEvent>(
```

```
new ExceptionCatcherEventProcessor<MyEvent>(
    new SequentialEventProcessor<MyEvent>(handlers)),
    logger);
```

This creates a processor that handles events sequentially, catches any exceptions, and logs every outcome.

Fl.Event.Handling.Extensions

This library provides dependency injection extensions for registering the event handling components from [Fl.Event.Handling](#) into an `IServiceCollection`.

Components

EventHandlingStrategy

An enum that controls how an event is dispatched to its registered handlers.

Value	Description
<code>Sequential</code>	Handlers are invoked one at a time, in registration order. Processing stops immediately if any handler returns an error.
<code>Parallel</code>	Handlers are invoked concurrently and all are awaited before the result is returned.

ServiceCollectionExtensions

Provides the `AddEventHandling<TEvent>` extension method on `IServiceCollection`.

```
IServiceCollection AddEventHandling<TEvent>(  
    this IServiceCollection services,  
    EventHandlingStrategy strategy,  
    Assembly assembly)  
where TEvent : class
```

This method:

1. Registers an `IEventProcessor<TEvent>` singleton — either `SequentialEventProcessor<TEvent>` or `ParallelEventProcessor<TEvent>` — depending on the chosen `strategy`.
2. Scans the provided `assembly` and registers all concrete `IEventHandler<TEvent>` implementations as singleton services.

An `ArgumentException` is thrown if `strategy` is not a valid `EventHandlingStrategy` value.

Usage

```
using Fl.Event.Handling.Extensions;
using System.Reflection;

// Inside your IServiceCollection configuration (e.g. Program.cs or a
startup class):
services.AddEventHandling<MyEvent>(
    EventHandlingStrategy.Sequential,
    Assembly.GetExecutingAssembly());
```

To use the parallel strategy instead:

```
services.AddEventHandling<MyEvent>(
    EventHandlingStrategy.Parallel,
    Assembly.GetExecutingAssembly());
```

Multiple event types can be registered independently:

```
services.AddEventHandling<OrderPlacedEvent>(EventHandlingStrategy.Sequential,
    Assembly.GetExecutingAssembly());
services.AddEventHandling<PaymentReceivedEvent>(EventHandlingStrategy.Parallel,
    Assembly.GetExecutingAssembly());
```

Each call scans the assembly for the corresponding `IEventHandler<TEvent>` implementations and registers them alongside their processor.